Assignment 9: ML Pipeline Orchestration with Apache Airflow

1. Objective

The goal of this assignment is to automate a complete machine learning workflow with Apache Airflow. Instead of running notebook cells by hand, I defined each step of the workflow as a separate task inside a DAG (Directed Acyclic Graph). Airflow runs the tasks in the right order, shows their status in a web interface, keeps a log for every task, and retries a task automatically if it fails. My workflow loads the Iris flower dataset, cleans it, splits it, saves the splits, trains a classifier and evaluates it.

2. Environment and setup

I ran Airflow inside Docker on my Windows laptop, using the official apache/airflow:latest image (Airflow 3) in standalone mode. Standalone mode starts the scheduler, the API server (the web UI), the DAG processor and a local metadata database in one container, which is ideal for learning and testing. I mounted two folders from my laptop into the container. The dags folder holds my pipeline code, which Airflow scans automatically. The data folder is where the tasks write their files, so I can see the results on my own computer. scikit-learn was installed at start-up through the _PIP_ADDITIONAL_REQUIREMENTS setting. I started Airflow with this command from Git Bash:

```
MSYS_NO_PATHCONV=1 docker run -d --name airflow -p 8080:8080 \
  -v "$(pwd -W)/dags:/opt/airflow/dags" \
  -v "$(pwd -W)/data:/opt/airflow/data" \
  -e _PIP_ADDITIONAL_REQUIREMENTS="scikit-learn pandas" \
  -e AIRFLOW__CORE__LOAD_EXAMPLES=False \
  apache/airflow:latest standalone
```

Then I opened the Airflow UI at http://localhost:8080 and logged in as admin with the password that Airflow printed in the container logs.

3. Pipeline design (DAG)

The DAG is called iris_ml_pipeline. It has six tasks connected in a straight line: load_data → clean_data → split_data → save_data → train_model → evaluate_model. Each task is a PythonOperator that calls one Python function. The DAG is triggered manually (schedule=None), and each task retries once after one minute if it fails. This is what each task does:

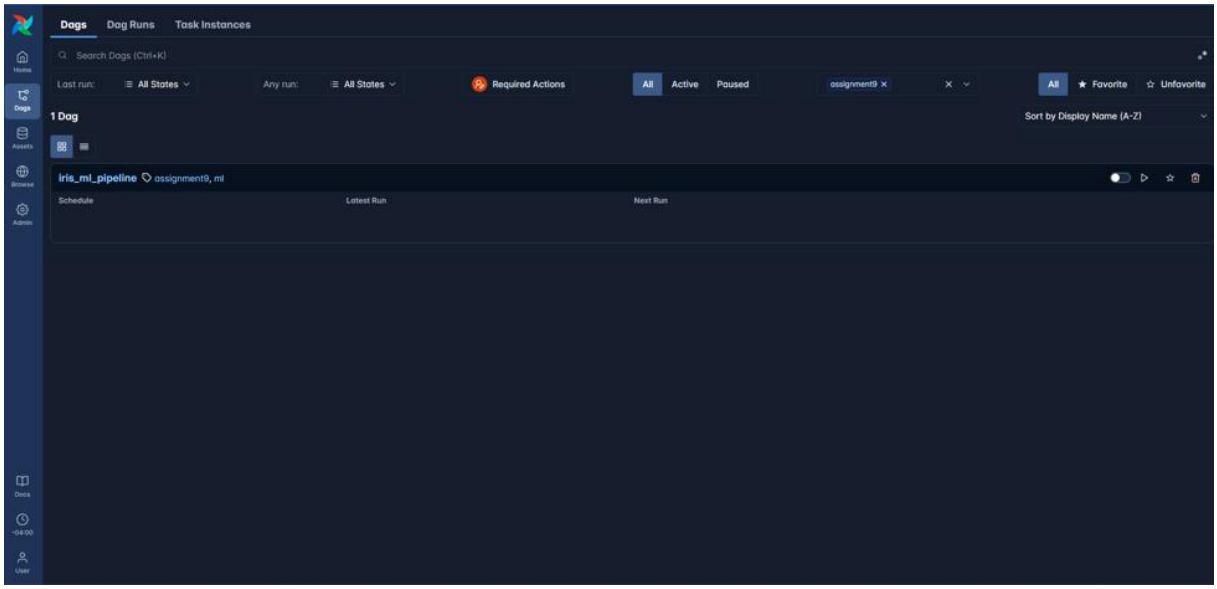
- load_data: loads the Iris dataset from scikit-learn (150 rows, 4 measurements and the species) and saves it as iris_raw.csv.

- clean_data: removes duplicate rows and missing values, checks that all measurements are positive, and saves iris_clean.csv (149 rows).
- split_data: makes a stratified 80/20 train/test split (119 and 30 rows) and passes both parts to the next task using XCom.
- save_data: takes the split from XCom and saves it as train.csv and test.csv.
- train_model: trains a StandardScaler + Logistic Regression pipeline and saves it as model.pkl with pickle.
- evaluate_model: tests the model on unseen data, prints a classification report, saves metrics.json, and fails the run on purpose if accuracy is below 0.80 (a quality gate).

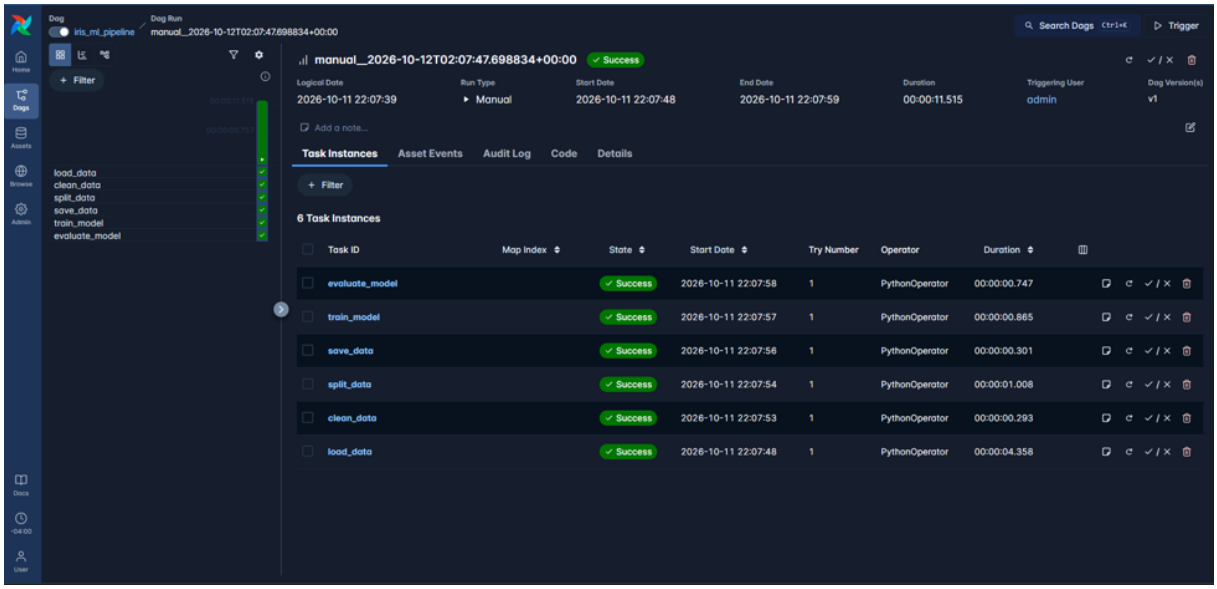
How data moves between tasks: every Airflow task runs as its own separate process, so tasks cannot share Python variables. I used two methods to pass data. Most tasks read and write CSV files in the shared data folder. The split step uses XCom (cross-communication), which is Airflow's built-in way to pass small pieces of data between tasks. XCom is fine for a small dataset like Iris, but for large data, files or a database are the better choice.

4. Running the pipeline

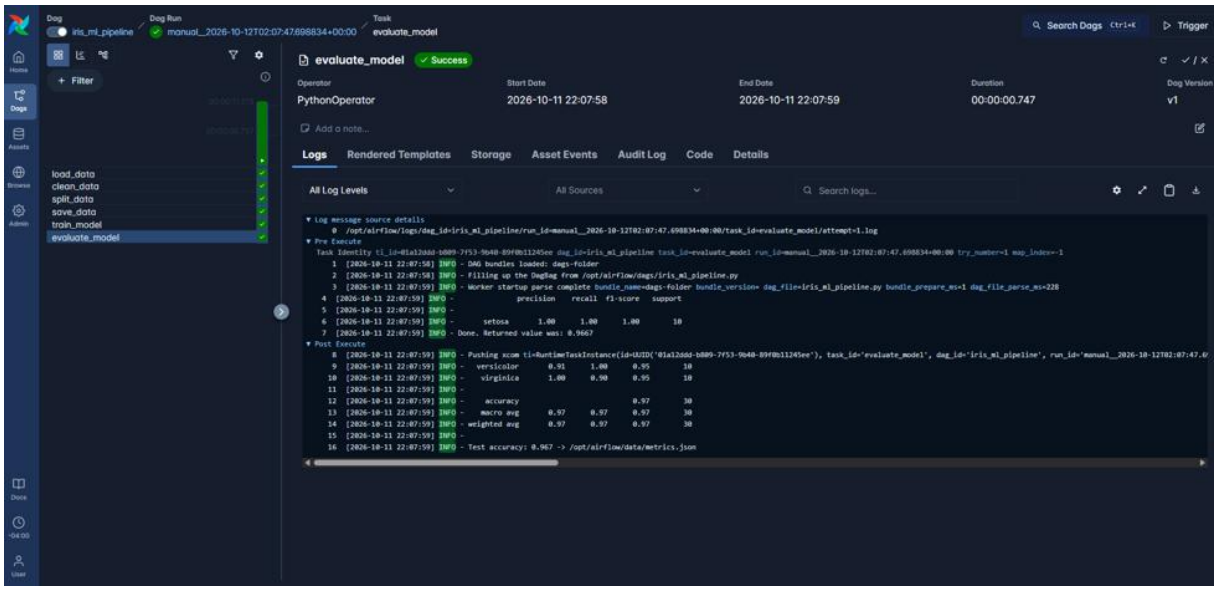
After I placed the DAG file in the dags folder, Airflow detected it automatically and listed it on the Dags page with my tags assignment9 and ml.



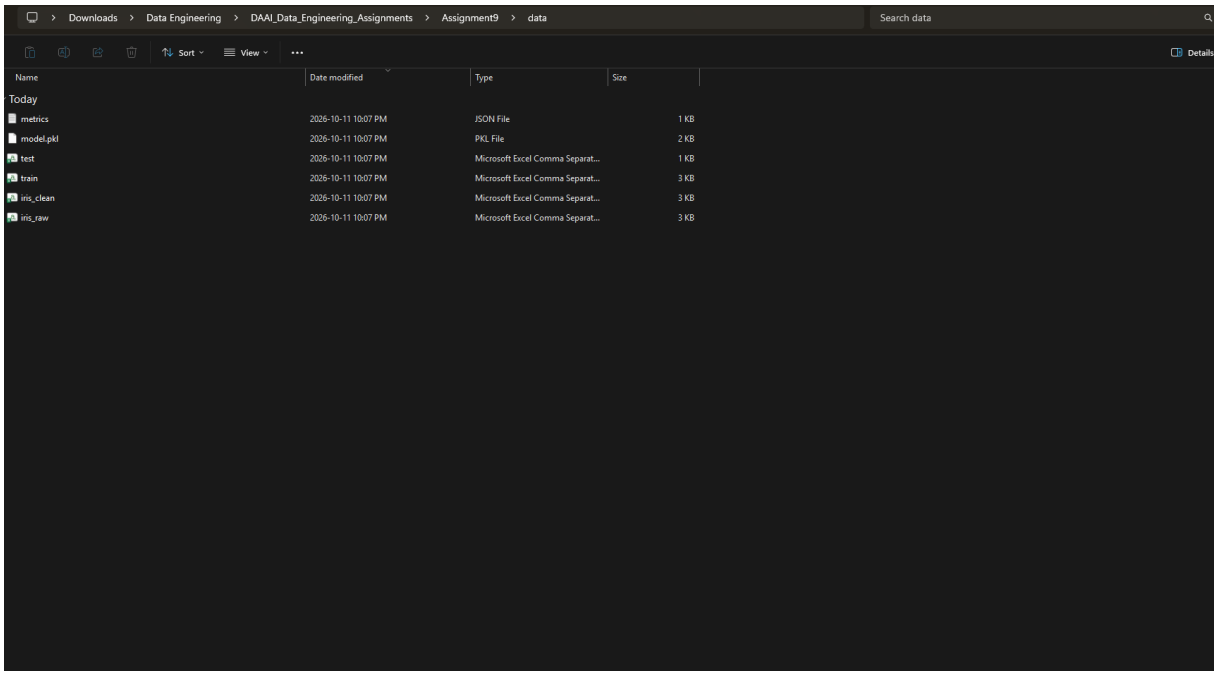
I triggered a run manually with the Trigger button. The run page shows the six tasks in the grid on the left and the task list on the right. All six tasks finished with status Success on the first try. The whole run took 11.5 seconds. load_data was the slowest at 4.4 seconds, because it imports scikit-learn for the first time, and every other task took less than 1.1 seconds.



The log of the evaluate_model task shows the classification report printed by my code, and the final line confirms that the metrics were saved.



The tasks wrote all their outputs to the mounted data folder on my laptop.



5. Results

The cleaning step removed 1 duplicate row, so the data went from 150 to 149 rows. The stratified split produced 119 training rows and 30 test rows, with 10 test flowers of each species. The Logistic Regression model reached a test accuracy of 0.967 and a macro F1-score of 0.967. All setosa and versicolor flowers were classified correctly, and only 1 virginica flower was predicted as versicolor. This makes sense, because virginica and versicolor overlap a little in petal size, while setosa is completely different from the other two. The accuracy is well above the 0.80 quality gate, so Airflow marked the run as successful.

6. Challenges and how I solved them

Docker engine not running: my first docker pull failed with the message "failed to connect to the docker API". The fix was simply to open Docker Desktop and wait until it showed "Engine running".

Windows paths in Git Bash: Git Bash changes paths that start with a slash, which breaks Docker volume mounts. I used MSYS_NO_PATHCONV=1 and $(pwd -W) so Docker receives correct Windows paths.

Missing library inside the container: the Airflow image does not include scikit-learn, so I installed it at start-up with _PIP_ADDITIONAL_REQUIREMENTS. For production, building a custom Docker image is the better approach, because it does not reinstall packages on every start.

Passing data between tasks: tasks do not share memory, so I used files and XCom. During testing, passing the data through XCom as a JSON string failed with the newer pandas version, so I passed the rows as a list of records instead.

Airflow 3 changes: Airflow 3 moved the imports to airflow.sdk and airflow.providers.standard. My DAG tries the new imports first and falls back to the old Airflow 2 paths, so the same file works on both versions.

7. Conclusion

Airflow turned a manual notebook workflow into a repeatable, monitored pipeline. Each step is isolated, so if one step fails I can see exactly which one in the grid view, read its log, and re-run only that step instead of the whole workflow. Retries and the accuracy quality gate make the pipeline safer for production, because a weak model is stopped before anyone uses it. Natural next steps would be:

- add a schedule, for example daily retraining
- add a data-validation step and a drift-detection step like the one in Assignment 7
- add a deployment step that copies the new model into the Flask API from Assignment 5
- store the data and models in cloud storage instead of a local folder

Together, Assignments 5, 7, 8 and 9 make a complete MLOps (Machine Learning Operations) loop: serve the model, monitor it, retrain it, and orchestrate everything automatically.